

# Docker Containerization and Deployment of ElectroGO on Microsoft Azure

## Table of Contents

1. Introduction
2. Objective
3. Prerequisites
4. Phase 1 – Azure Infrastructure Setup
5. Phase 2 – Create Docker Files
6. Phase 3 – Install Docker on Azure VM
7. Phase 4 – Build Docker Image and Push to Azure Container Registry
8. Phase 5 – Deploy Two-Tier Application using Docker Compose
9. Phase 6 – Verification
10. Conclusion

# Docker Containerization and Deployment of ElectroGO on Microsoft Azure

## Introduction

This project demonstrates how to containerize an ASP.NET Core application using Docker and deploy it on Microsoft Azure. The deployment uses Azure Container Registry (ACR), an Ubuntu Virtual Machine, Docker Compose, and Managed Identity for secure authentication without storing registry credentials.

## Objective

The objectives of this project are:

- Create Azure infrastructure
- Create Dockerfile and Docker Compose configuration
- Build Docker images
- Push Docker images to Azure Container Registry
- Deploy a two-tier application using Docker Compose

- Use Managed Identity for password-less ACR authentication
- Verify successful deployment

## Prerequisites

- Azure Subscription
- Azure DevOps Repository
- Ubuntu 22.04 Virtual Machine
- Azure Container Registry
- Docker
- Docker Compose
- Azure CLI

## Phase 1 – Azure Infrastructure Setup

### 1.1 Create Resource Group

Steps:

- Open Azure Portal.
- Navigate to **Resource Groups**.
- Click **Create**.
- Resource Group Name: **ElectroGo-Rg**
- Region: **Austria East**
- Click **Review + Create**
- Click **Create**

#### Screenshot

The screenshot shows the Azure Portal interface for Resource Groups. The top navigation bar includes the search bar, Copilot, and user profile. The main content area is titled 'Resource groups' and includes a filter bar with options like 'Export resource groups using Bicep or Terraform', 'Summarize my costs by service', and 'How to manage changes with deployment tools?'. Below the filter bar, there are tabs for 'Create', 'Manage view', 'Refresh', 'Export to CSV', 'Open query', 'Assign tags', and 'Add to service group'. The main table lists resource groups with columns for Name, Subscription, and Location. The 'ElectroGo-Rg' resource group is highlighted.

| Name             | Subscription         | Location       |
|------------------|----------------------|----------------|
| agritech-rg      | Azure subscription 1 | Korea Central  |
| ElectroGo-Rg     | Azure subscription 1 | Austria East   |
| NetworkWatcherRG | Azure subscription 1 | Australia East |

## 1.2 Create Azure Container Registry

Steps:

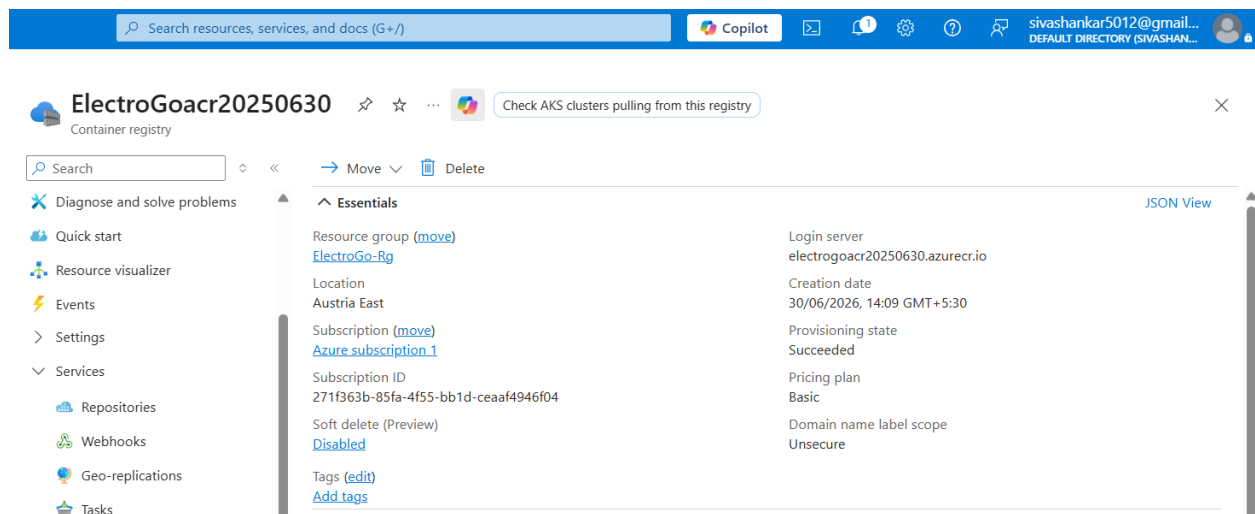
- Search **Container Registries**
- Click **Create**
- Registry Name: **electrogoacrxxxx**
- SKU: **Basic**
- Region: **Austria East**
- Click **Review + Create**

After deployment:

- Open **Access Keys**
- Verify **Admin User = Disabled**
- Copy the Login Server

Example:

electrogoacr.azurecr.io



## 1.3 Create Ubuntu Virtual Machine

Configuration

| Setting        | Value            |
|----------------|------------------|
| VM Name        | electrogo-vm     |
| OS             | Ubuntu 22.04 LTS |
| Size           | Standard_B2s     |
| Authentication | SSH Public Key   |

Katta siva shankar

Sivashankar9929@gmail.com

|            |                     |
|------------|---------------------|
| Username   | azureuser           |
| Open Ports | SSH (22), HTTP (80) |

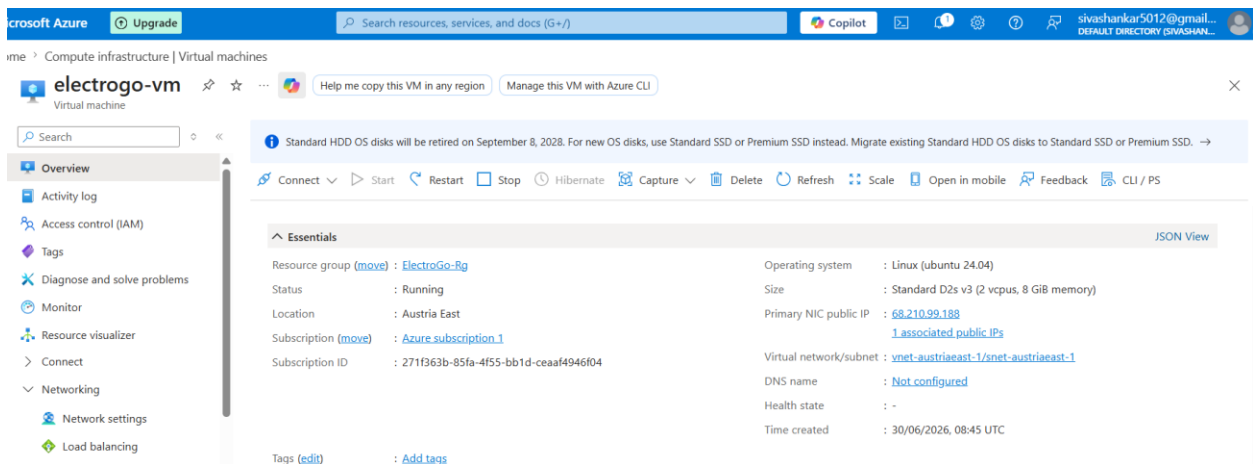
Download:

electrogo-key.pem

Copy:

Public IP Address

### Screenshot



The screenshot shows the Azure portal interface for a virtual machine named 'electrogo-vm'. The left sidebar contains navigation options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Monitor, Resource visualizer, Connect, Networking, Network settings, and Load balancing. The main pane displays the 'Overview' tab for the VM, showing its status as 'Running' and location as 'Austria East'. It also lists the operating system as 'Linux (ubuntu 24.04)' and the size as 'Standard D2s v3 (2 vcpus, 8 GiB memory)'. The primary NIC public IP is '68.210.99.188'. A warning banner at the top indicates that Standard HDD OS disks will be retired on September 8, 2028, and suggests migrating to Standard SSD or Premium SSD.

## 1.4 Configure Networking

Verify:

- Port 80
- Port 22

are allowed in the NSG.

### Screenshot

Network security group **electrogo-vm-nsg** (attached to networkInterface: **electrogo-vm894**)  
Impacts 0 subnets, 1 network interfaces

[+ Create port rule](#)

Search rules

Source == all Destination == all Protocol == all Action == all Port == all

| Priority ↑             | Name                          | Port | Protocol | Source            | Destination    | Action |
|------------------------|-------------------------------|------|----------|-------------------|----------------|--------|
| Inbound port rules (8) |                               |      |          |                   |                |        |
| 300                    | HTTP                          | 80   | TCP      | Any               | Any            | Allow  |
| 320                    | SSH                           | 22   | TCP      | Any               | Any            | Allow  |
| 330                    | https                         | 443  | TCP      | Any               | Any            | Allow  |
| 340                    | asp.net                       | 5000 | TCP      | Any               | Any            | Allow  |
| 350                    | mysql                         | 3306 | TCP      | Any               | Any            | Allow  |
| 65000                  | AllowVnetInBound              | Any  | Any      | VirtualNetwork    | VirtualNetwork | Allow  |
| 65001                  | AllowAzureLoadBalancerInBound | Any  | Any      | AzureLoadBalancer | Any            | Allow  |
| 65500                  | DenyAllInBound                | Any  | Any      | Any               | Any            | Deny   |

1.5 Enable Managed Identity

Steps

- VM
- Identity
- System Assigned
- Turn ON
- Save

Screenshot

Microsoft Azure

Upgrade

Search resources, services, and docs (G+)

Copilot

svashankar5012@gmail...  
DEFAULT DIRECTORY (SVASHAN...

me > Compute infrastructure | Virtual machines > electrogo-vm

electrogo-vm | Identity

Virtual machine

Search

Networking

Settings

Disks

Extensions + applications

Operating system

Configuration

Advisor recommendations

Properties

Locks

System assigned User assigned

A system assigned managed identity is restricted to one per resource and is tied to the lifecycle of this resource. You can grant permissions to the managed identity by using Azure role-based access control (Azure RBAC). The managed identity is authenticated with Microsoft Entra ID, so you don't have to store any credentials in code.

Save Discard Refresh Got feedback?

Status

Off On

Object (principal) ID

6acbd9fe-998b-4b69-a5a0-febace91ca37

Permissions

Azure role assignments

1.6 Assign Azure Container Registry Permissions

Navigate to:

Container Registry  
→ Access Control (IAM)  
→ Add Role Assignment

Assign:

Role:  
AcrPush

Member:  
electrogo-vm

#### Screenshot

### katta siva shankar assignments - ElectroGoacr20250630



Current role assignments   Eligible assignments

Assignments for the selected user, group, service principal, or managed identity at this scope or inherited to this scope.

 Search by assignment name or description

Role assignments (2) ⓘ

| Role  | Description                           | Scope                    | Group assignment | Condition |
|-------|---------------------------------------|--------------------------|------------------|-----------|
| Owner | Grants full access to manage all r... | Subscription (Inherited) | --               | None      |
| Owner | Grants full access to manage all r... | Subscription (Inherited) | --               | None      |

## Phase 2 – Docker File Creation

Three Docker-related files were added to the Azure DevOps repository.

### Dockerfile

The Dockerfile uses a multi-stage build.

Stages:

- Build Stage
- Runtime Stage

Features:

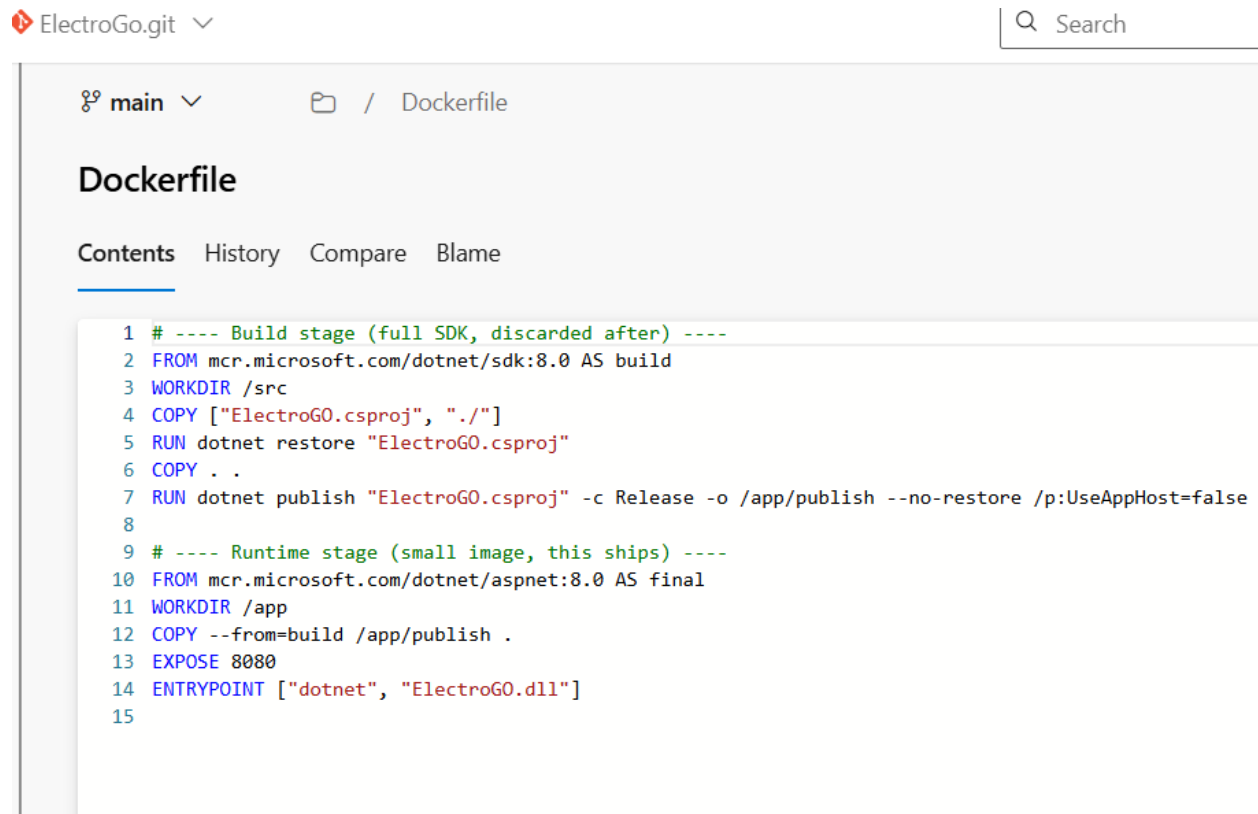
- .NET SDK Image
- ASP.NET Runtime Image
- Publish Application

**Katta siva shankar**

**Sivashankar9929@gmail.com**

- Expose Port 8080

### Screenshot

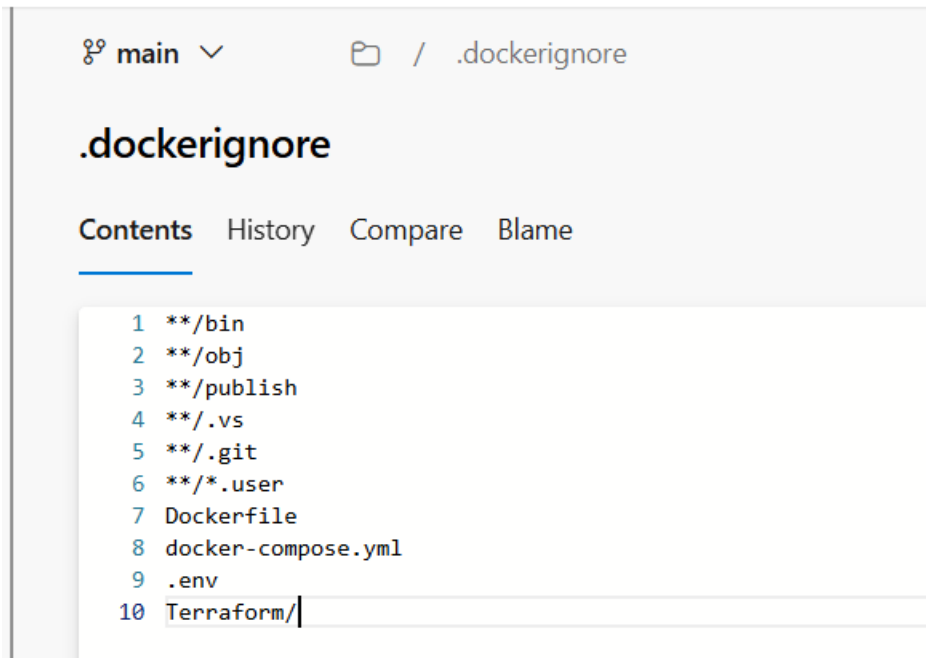


### *.dockerignore*

Ignored folders:

- bin
- obj
- publish
- .git
- .vs

### Screenshot



The screenshot shows a web-based Git interface for a repository named 'ElectroGo.git'. The current branch is 'main'. The file path is '/ .dockerignore'. The file is titled '.dockerignore'. Below the title are tabs for 'Contents', 'History', 'Compare', and 'Blame'. The 'Contents' tab is selected, showing a list of files in the repository:

```
1 **/bin
2 **/obj
3 **/publish
4 **/.vs
5 **/.git
6 **/*.user
7 Dockerfile
8 docker-compose.yml
9 .env
10 Terraform/
```

## docker-compose.yml

The compose file contains two services.

### Service 1

- MySQL Database

### Service 2

- ASP.NET Core Application

### Features

- Environment Variables
- Persistent Volume
- Health Check
- Container Restart Policy

### Screenshot

```
1 version: "3.9"
2
3 services:
4   db:
5     image: mysql:8.0
6     container_name: electrogo-db
7     restart: unless-stopped
8     environment:
9       MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
10      MYSQL_DATABASE: ${MYSQL_DATABASE}
11      MYSQL_USER: ${MYSQL_USER}
12      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
13    volumes:
14      - db-data:/var/lib/mysql
15      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
16    healthcheck:
17      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
18      interval: 10s
19      timeout: 5s
20      retries: 10
21
22   app:
23     image: electrogoacr20250630.azurecr.io/electrogo:${IMAGE_TAG}
24     container_name: electrogo-app
25     restart: unless-stopped
26     depends_on:
27       db:
28         condition: service_healthy
29     environment:
30       ASPNETCORE_ENVIRONMENT: Production
31       ASPNETCORE_HTTP_PORTS: "8080"
32       ConnectionStrings__DefaultConnection: "Server=db;Port=3306;Database=${MYSQL_DATABASE};User=${MYSQL_USER};Password=${MYSQL_PASS
```

## Phase 3 – Docker Installation on Ubuntu VM

SSH into VM

- `ssh azureuser@<VM_IP>`

Install Docker

- `curl -fsSL https://get.docker.com | sudo sh`

Enable Docker

- `sudo systemctl enable --now docker`

Install Azure CLI

- `curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash`

Verify installation

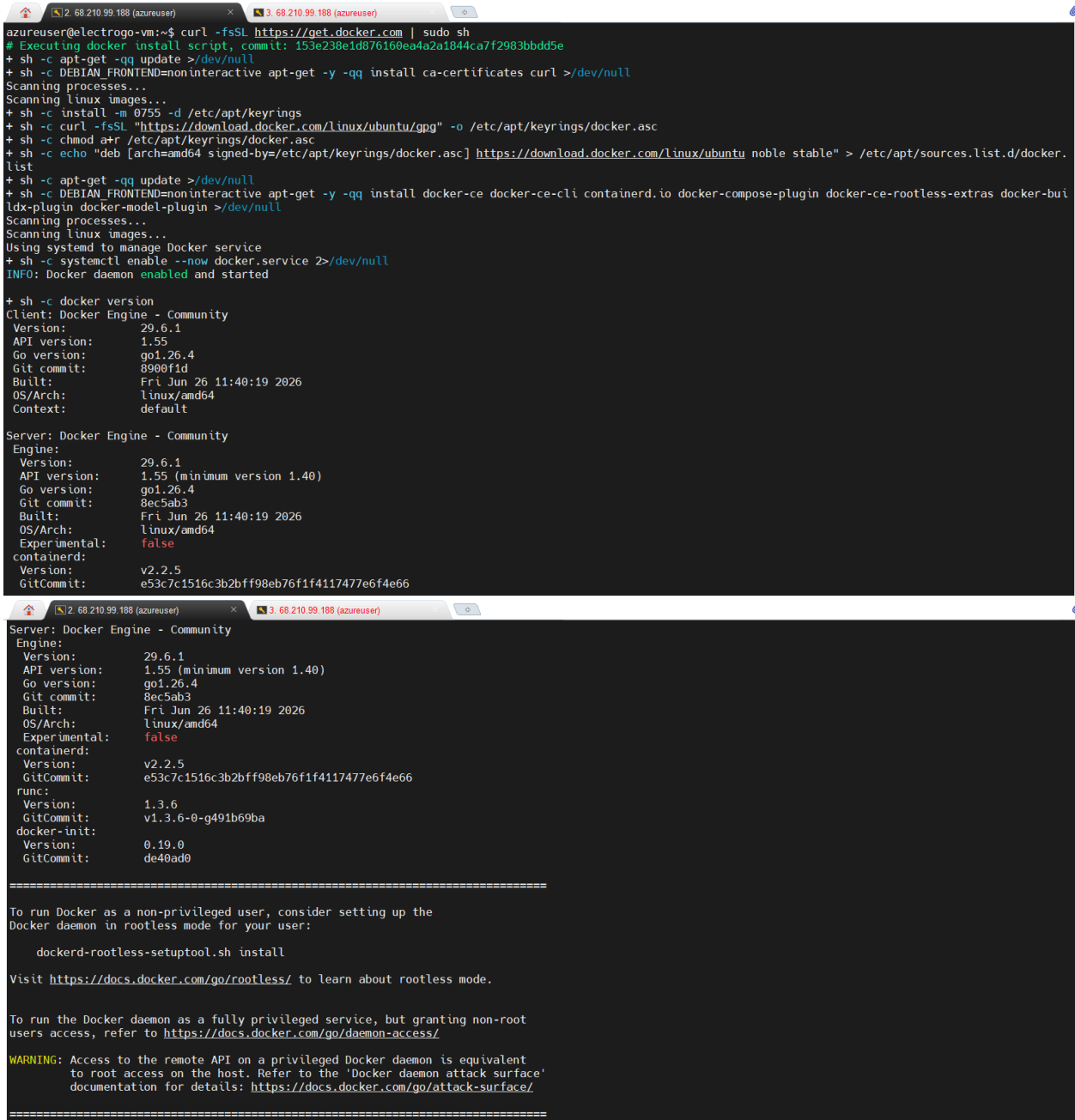
- `docker --version`

Katta siva shankar

Sivashankar9929@gmail.com

- docker compose version

## Screenshot



```
azureuser@electrogo-vm:~$ curl -fsSL https://get.docker.com | sudo sh
# Executing docker install script, commit: 153e238e1d876160ea4a2a1844ca7f2983bbdd5e
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install ca-certificates curl >/dev/null
Scanning processes...
Scanning linux images...
+ sh -c install -m 0755 -d /etc/apt/keyrings
+ sh -c curl -fsSL "https://download.docker.com/linux/ubuntu/gpg" -o /etc/apt/keyrings/docker.asc
+ sh -c chmod a+r /etc/apt/keyrings/docker.asc
+ sh -c echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu noble stable" > /etc/apt/sources.list.d/docker.list
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io docker-compose-plugin docker-ce-rootless-extras docker-buildx-plugin docker-model-plugin >/dev/null
Scanning processes...
Scanning linux images...
Using systemd to manage Docker service
+ sh -c systemctl enable --now docker.service 2>/dev/null
INFO: Docker daemon enabled and started

+ sh -c docker version
Client: Docker Engine - Community
Version:           29.6.1
API version:       1.55
Go version:        go1.26.4
Git commit:        8900f1d
Built:             Fri Jun 26 11:40:19 2026
OS/Arch:           linux/amd64
Context:           default

Server: Docker Engine - Community
Engine:
Version:           29.6.1
API version:       1.55 (minimum version 1.40)
Go version:        go1.26.4
Git commit:        8ec5ab3
Built:             Fri Jun 26 11:40:19 2026
OS/Arch:           linux/amd64
Experimental:      false
containerd:
Version:           v2.2.5
GitCommit:         e53c7c1516c3b2bff98eb76f1f4117477e6f4e66

Server: Docker Engine - Community
Engine:
Version:           29.6.1
API version:       1.55 (minimum version 1.40)
Go version:        go1.26.4
Git commit:        8ec5ab3
Built:             Fri Jun 26 11:40:19 2026
OS/Arch:           linux/amd64
Experimental:      false
containerd:
Version:           v2.2.5
GitCommit:         e53c7c1516c3b2bff98eb76f1f4117477e6f4e66
runc:
Version:           1.3.6
GitCommit:         v1.3.6-0-g491b69ba
docker-init:
Version:           0.19.0
GitCommit:         de40ad0

=====

To run Docker as a non-privileged user, consider setting up the
Docker daemon in rootless mode for your user:

    dockerd-rootless-setuptool.sh install

Visit https://docs.docker.com/go/rootless/ to learn about rootless mode.

To run the Docker daemon as a fully privileged service, but granting non-root
users access, refer to https://docs.docker.com/go/daemon-access/

WARNING: Access to the remote API on a privileged Docker daemon is equivalent
to root access on the host. Refer to the 'Docker daemon attack surface'
documentation for details: https://docs.docker.com/go/attack-surface/

=====
```

## Screenshot

```
azureuser@electrogo-vm:~$ docker --version
Docker version 29.6.1, build 8900f1d
azureuser@electrogo-vm:~$ docker compose version
Docker Compose version v5.2.0
azureuser@electrogo-vm:~$ az login --identity
[
  {
    "environmentName": "AzureCloud",
    "homeTenantId": "7276fb1d-241b-4606-bf7a-02455aa9580d",
    "id": "271f363b-85fa-4f55-bb1d-ceaaf4946f04",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Azure subscription 1",
    "state": "Enabled",
    "tenantId": "7276fb1d-241b-4606-bf7a-02455aa9580d",
    "user": {
      "assignedIdentityInfo": "MSI",
      "name": "systemAssignedIdentity",
      "type": "servicePrincipal"
    }
  }
]
```

## Phase 4 – Build Docker Image and Push to Azure Container Registry

### Login using Managed Identity

- `az login --identity`

Login to ACR

- `az acr login --name <acr-name>`

 **Screenshot**



## Push Docker Image

```
docker push <acr>.azurecr.io/electrogo:v1
```

```
docker push <acr>.azurecr.io/electrogo:latest
```

### Screenshot

```
azureuser@electrogo-vm:~/ElectroGo$ docker push electrogoacr20250630.azurecr.io/electrogo:v1
The push refers to repository [electrogoacr20250630.azurecr.io/electrogo]
51e77fdebcd9: Pushed
065b14c25713: Pushed
7c45f8d6397c: Pushed
68629629b516: Pushed
d4a653bde021: Pushed
41055cb5e9c1: Pushed
4836098c2553: Pushed
5158e9e18754: Pushed
a10fc6bd1045: Pushed
v1: digest: sha256:a50fbfa88df40774f31eaf2cfffdba5e0a60e0e42eb54b9db49ab535a66f1275 size: 856
azureuser@electrogo-vm:~/ElectroGo$ docker push electrogoacr20250630.azurecr.io/electrogo:latest
The push refers to repository [electrogoacr20250630.azurecr.io/electrogo]
4836098c2553: Layer already exists
065b14c25713: Layer already exists
d4a653bde021: Layer already exists
a10fc6bd1045: Layer already exists
5158e9e18754: Layer already exists
51e77fdebcd9: Already exists
68629629b516: Layer already exists
41055cb5e9c1: Layer already exists
7c45f8d6397c: Layer already exists
latest: digest: sha256:a50fbfa88df40774f31eaf2cfffdba5e0a60e0e42eb54b9db49ab535a66f1275 size: 856
azureuser@electrogo-vm:~/ElectroGo$ pwd
/home/azureuser/ElectroGo
azureuser@electrogo-vm:~/ElectroGo$ ls
```

## Verify Repository

Azure Portal

Container Registry

↓

Repositories

↓

electrogo

↓

Katta siva shankar

Sivashankar9929@gmail.com

latest

v1

 **Screenshot 18: ACR Repository**

## Phase 5 – Deploy Two-Tier Application

Create environment file

```
cat > .env <<EOF
```

Run application

```
docker compose pull
```

```
docker compose up -d
```

Verify containers

```
docker compose ps
```

Expected Containers

- electrogo-app
- electrogo-db

 **Screenshot**

```
azureuser@electrogo-vm:~/ElectroGo$ docker compose pull
WARN[0000] /home/azureuser/ElectroGo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
n
[+] pull 15/15
✓ Image electrogoacr20250630.azurecr.io/electrogo:v1 Pulled 0.5s
✓ Image mysql:8.0 Pulled 22.7s
azureuser@electrogo-vm:~/ElectroGo$ docker compose up -d
WARN[0000] /home/azureuser/ElectroGo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
n
[+] up 4/4
✓ Network electrogo_default Created 0.1s
✓ Volume electrogo_db-data Created 0.0s
✓ Container electrogo-db Healthy 34.2s
✓ Container electrogo-app Started 31.6s
```

 **Screenshot**

```
azureuser@electrogo-vm:~/ElectroGo$ docker compose ps
WARN[0000] /home/azureuser/ElectroGo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
n
NAME                IMAGE                                COMMAND                                SERVICE    CREATED         STATUS           PORTS
electrogo-app        electrogoacr20250630.azurecr.io/electrogo:v1  "dotnet ElectroGo.dll"                app        About a minute ago    Restarting (139) 6 seconds ago
electrogo-db         mysql:8.0                                "docker-entrypoint.s..."            db         About a minute ago    Up About a minute (healthy)    3306/tcp
```

## View Logs

docker compose logs app

### Screenshot 21: Application Logs

```
azureuser@electrogo-vm:~/ElectroGo$ docker compose logs app
WARN[0000] /home/azureuser/ElectroGo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
electrogo-app | Unhandled exception. System.IO.InvalidDataException: Failed to load configuration from file '/app/appsettings.json'.
electrogo-app | ---> System.FormatException: Could not parse the JSON file.
electrogo-app | ---> System.Text.Json.JsonReaderException: ':' is invalid after a single JSON value. Expected end of data. LineNumber: 0 | BytePositionInLine: 19.
electrogo-app |    at System.Text.Json.ThrowHelper.ThrowJsonReaderException(Utf8JsonReader& json, ExceptionResource resource, Byte nextByte, ReadOnlySpan`1 bytes)
electrogo-app |    at System.Text.Json.Utf8JsonReader.ConsumeNextTokenUntilAfterAllCommentsAreSkipped(Byte marker)
electrogo-app |    at System.Text.Json.Utf8JsonReader.ConsumeNextToken(Byte marker)
electrogo-app |    at System.Text.Json.Utf8JsonReader.ConsumeNextTokenOrRollback(Byte marker)
electrogo-app |    at System.Text.Json.Utf8JsonReader.ReadSingleSegment()
electrogo-app |    at System.Text.Json.Utf8JsonReader.Read()
electrogo-app |    at System.Text.Json.JsonDocument.Parse(ReadOnlySpan`1 utf8JsonSpan, JsonReaderOptions readerOptions, MetadataDb& database, StackRowStack& stack)
electrogo-app |    at System.Text.Json.JsonDocument.Parse(ReadOnlyMemory`1 utf8Json, JsonReaderOptions readerOptions, Byte[] extraRentedArrayPoolBytes, PoolableByteBufferWriter extraPooledByteBufferWriter)
electrogo-app |    at System.Text.Json.JsonDocument.Parse(ReadOnlyMemory`1 json, JsonDocumentOptions options)
electrogo-app |    at Microsoft.Extensions.Configuration.Json.JsonConfigurationFileParser.ParseStream(Stream input)
electrogo-app |    at Microsoft.Extensions.Configuration.Json.JsonConfigurationProvider.Load(Stream stream)
electrogo-app |    --- End of inner exception stack trace ---
electrogo-app |    at Microsoft.Extensions.Configuration.Json.JsonConfigurationProvider.Load(Stream stream)
electrogo-app |    at Microsoft.Extensions.Configuration.FileConfigurationProvider.Load(Boolean reload)
electrogo-app |    --- End of inner exception stack trace ---
electrogo-app |    at Microsoft.Extensions.Configuration.FileConfigurationProvider.Load(Boolean reload)
electrogo-app |    at Microsoft.Extensions.Configuration.ConfigurationManager.AddSource(IConfigurationSource source)
electrogo-app |    at Microsoft.Extensions.Configuration.ConfigurationManager.Microsoft.Extensions.Configuration.IConfigurationBuilder.Add(IConfigurationSource source)
electrogo-app |    at Microsoft.Extensions.Hosting.HostingHostBuilderExtensions.ApplyDefaultAppConfiguration(HostBuilderContext hostingContext, IConfigurationBuilder appConfigBuilder, String[] args)
electrogo-app |    at Microsoft.Extensions.Hosting.HostApplicationBuilder..ctor(HostApplicationBuilderSettings settings)
electrogo-app |    at Microsoft.AspNetCore.Builder.WebApplicationBuilder..ctor(WebApplicationOptions options, Action`1 configureDefaults)
electrogo-app |    at Microsoft.AspNetCore.Builder.WebApplicationBuilder.CreateBuilder(String[] args)
electrogo-app |    at Program.<Main>$(String[] args) in /src/Program.cs:line 7
```

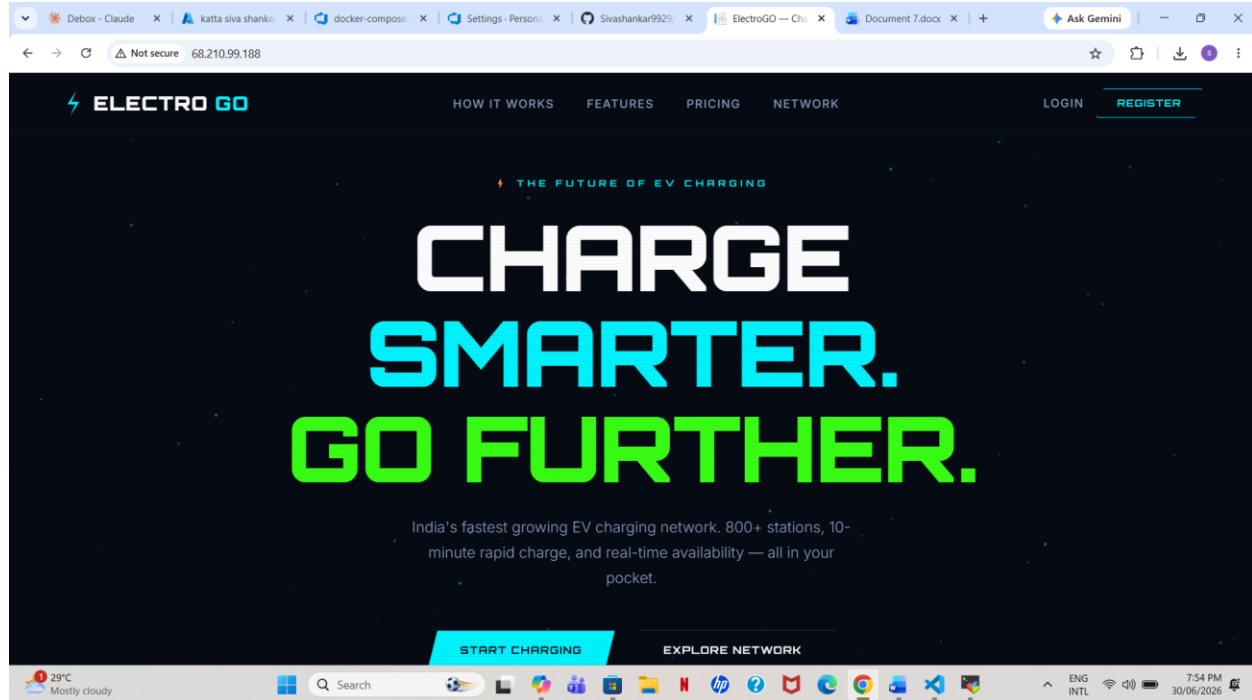
## Phase 6 – Verification

Open Browser

<http://<VM IP>>

The ElectroGO application should load successfully.

### Screenshot 22: Running ElectroGO Website



## Project Outcome

Successfully completed:

- ✓ Azure Infrastructure Deployment
- ✓ Docker Containerization
- ✓ Azure Container Registry Integration
- ✓ Managed Identity Authentication
- ✓ Docker Image Build
- ✓ Docker Image Push
- ✓ Docker Compose Deployment
- ✓ Two-Tier Application Deployment
- ✓ ASP.NET Core Application Running Successfully

Katta siva shankar

Sivashankar9929@gmail.com

## **Conclusion**

The ElectroGO application was successfully containerized using Docker and deployed on an Azure Ubuntu Virtual Machine. Docker images were securely stored in Azure Container Registry, and Managed Identity was used for password-less authentication. Docker Compose orchestrated the ASP.NET Core application and MySQL database containers, resulting in a fully functional two-tier deployment accessible through the VM's public IP.